Claude Code Cheatsheet

L'essentiel au quotidien pour une productivité maximale

v3.44.1 | octobre 2026

Installation

```
curl -fsSL https://claude.ai/install.sh | bash
# Vérifier l'installation
which claude && claude --version
```

Contrôle santé : `claude doctor && claude mcp list`

Commandes essentielles

Commande	Action
`/help`	Aide contextuelle
`/powerup`	Leçons animées interactives sur les fonctionnalités de Claude Code
`/clear`	Réinitialiser la conversation
`/compact`	Libérer du contexte
`/status`	Onglet Status : version, modèle, compte, connectivité
`/context`	Détail de la consommation de tokens
`/plan`	Explorer sous les restrictions du mode plan
Approuver le plan	Sortir du Plan Mode (`Shift+Tab` pour sortir sans valider)
`/model`	Choisir le modèle ; Entrée enregistre, s limite à la session
`/insights`	Analytique d'usage + rapport d'optimisation
`/simplify`	Revue de nettoyage uniquement sur le code modifié : réutilisation, simplification, efficacité
`/batch`	Refactors à grande échelle via 5 à 30 agents en worktrees parallèles
`/teleport`	Téléporter une session depuis le web
`/tasks`	Suivre les tâches en arrière-plan
`/remote-env`	Configurer un environnement cloud
`/rc`	Démarrer une session de contrôle à distance (Pro/Max/ Team/Enterprise)
`/mobile`	Récupérer les liens de téléchargement de l'app mobile Claude
`/fast`	Basculer le mode rapide (Opus 5.5 : $8/$40 par MTok)
`/voice`	Activer la dictée vocale (maintenir ou taper Espace)
`/recap`	Résumé en une ligne de la session en cours, à la demande
`/effort [niveau]`	Profondeur de réflexion : low/ medium/high/xhigh/max
`/loop [intervalle] [prompt]`	Répète un prompt en boucle (rythme autonome sans intervalle) ; skill intégré
`/goal [condition]`	Mode multi-tours autonome jusqu'à ce que la condition soit remplie
`/rename [nom]`	Nommer ou renommer la session en cours
`/debug`	Active les journaux de débogage et diagnostique à partir du journal (skill intégré)
`/exit`	Quitter (alias `/quit` , ou Ctrl+D)

Raccourcis clavier

Raccourci	Action
`Shift+Tab`	Alterner les modes de permission
`Esc × 2`	Menu de rewind (invite vide) ou effacer le brouillon
`Ctrl+C`	Interrompre (au repos : vide la saisie, deux appuis pour quitter)

Raccourci	Action
`Ctrl+R`	Rechercher dans l'historique des commandes
`Ctrl+L`	Redessiner l'écran (garde le contexte)
`Ctrl+B`	Tâches en arrière-plan
`Ctrl+X Ctrl+K`	Tuer tous les agents en arrière-plan
`Alt+T`	Basculer la réflexion si le modèle le permet (pas sur Opus/Sonnet/Haiku 5.5, Fable)
`Espace` (maintenu)	Entrée vocale (nécessite `/voice`)
`Tab`	Accepter la suggestion d'autocomplétion
`Shift+Enter`	Nouvelle ligne
`Ctrl+D`	Quitter

Raccourcis IDE : VS Code `Alt+K` | JetBrains `Cmd+Option+K`

Références de fichiers

```
@chemin/vers/fichier.ts   → Référencer un fichier
Demander une délégation   → Claude peut utiliser un agent disponible
!commande-shell           → Exécuter une commande shell
```

Fonctionnalités méconnues (mais officielles !)

Fonctionnalité	Ce que ça fait
Tasks API	Listes persistantes avec dépendances ; activer `CLAUDE_CODE_ENABLE_TODO_TOOLS=` sur les modèles récents
Background Agents	Des sous-agents travaillent pendant que vous codez
Agent Teams	Coordination multi-agents (TeamCreate/SendMessage)
Auto-Memories	Activées par défaut ; inspecter et corriger avec `/memory`
Session Forking	`/branch` crée une branche de conversation ; `/fork` copie vers une session en arrière-plan
LSP Tool	Navigation vers définitions et références, selon les serveurs configurés
Mode vocal	Entrée vocale native, transcription gratuite
Remote Control	Contrôler une session locale depuis téléphone/navigateur (Pro/Max/Team/Enterprise)
Tâches planifiées (Cloud/Desktop)	Cloud : machine éteinte possible ; Desktop : app et machine actives
Skill Evals	Tests A/B et calibrage des déclencheurs pour les skills personnalisées
Output Styles	`/output-style` : Default, Proactive, Concise, Explanatory, Learning
Rules Templates	Templates `.claude/rules/` prêts à l'emploi (arch/qualité/tests/perf)

Astuce : ce ne sont pas des « secrets », tout est dans le CHANGELOG. Lisez-le !

Modes de permission

Mode	Édition	Exécution
Manual (`default`)	Demande	Demande
acceptEdits	Auto	Demande
Plan Mode	Aucune	Outils en lecture seule
auto	Le classifieur décide	Le classifieur décide
dontAsk	Uniquement si dans les règles autorisées	Uniquement si dans les règles autorisées
Bypass	Auto	Permissions ignorées ; nécessite une isolation et des accès adaptés

L'identifiant du mode Bypass est `bypassPermissions`.

Shift+Tab pour changer de mode. Depuis v2.1.283, les nouvelles sessions démarrent en mode auto sauf si `permissions.defaultMode` ou `--permission-mode` en décide autrement ; `/auto-mode-setup` guide la configuration.

Mémoire & configuration (2 niveaux)

Niveau	Chemin	Portée	Git
Projet	`.claude/`	Équipe	Oui
Personnel	`~/.claude/`	Vous (tous projets)	Non

Settings : politiques gérées > CLI > local projet > projet > utilisateur ; listes généralement fusionnées. Les instructions CLAUDE.md se combinent, sans priorité mécanique des textes.

Les préférences personnelles au projet peuvent être placées dans `CLAUDE.local.md` à la racine, ajouté à `.gitignore`. L'auto-memory native utilise `~/.claude/projects/<project>/memory/MEMORY.md` sur la machine.

Structure du dossier .claude/

```
.claude/
├── CLAUDE.md          # Instructions projet partagées
├── settings.json      # Hooks (versionné)
├── settings.local.json  # Permissions (non versionné)
├── agents/            # Agents personnalisés
├── hooks/             # Scripts d'événements
├── rules/             # Démarrage ou portée paths
│   ├── architecture-review.md
│   ├── code-quality-review.md
│   ├── test-review.md
│   └── performance-review.md
└── skills/            # Slash commands + modules de
connaissance (unifiés)
```

Gestion du contexte (CRITIQUE)

Statusline : `Model: Sonnet | Ctx: 89.5k | Ctx(u): 56.0%`

✅ 0-50%	⚠️ 50-70%	🔶 70-90%	🛑 90%+

Surveillez `/context` et la continuité du raisonnement. Sauvegardez les décisions avant compaction ou passation ; aucun seuil universel ne garantit une chute de qualité.

Signe	Action
Réponses courtes	`/compact`
Oublis fréquents	Vérifier les instructions, résumer l'état puis repartir si nécessaire
Contexte >70%	`/compact`
Tâche terminée	`/clear`

Commandes de récupération du contexte

Commande	Usage
`/compact`	Résumer et libérer du contexte
`/clear`	Repartir de zéro
`/rewind`	Revenir à un checkpoint ; ne révoque pas les effets Bash ou externes
`claude -c`	Reprendre la dernière session
`claude -r <id>`	Reprendre une session spécifique

Plan Mode & réflexion

Fonctionnalité	Activation	Usage
Plan Mode	`Shift+Tab × 2` ou `/plan`	Explorer sans modifier
OpusPlan	`/model opusplan`	Opus pour planifier, Sonnet pour exécuter

Thinking et effort

Opus 5.5, Sonnet 5.5, Haiku 5.5 et Fable 5.1 utilisent le thinking adaptatif. Haiku 4.5 (historique) n'a **pas de paramètre effort**.

Les modèles adaptatifs actuels acceptent `low`, `medium`, `high`, `xhigh` et `max`. Opus 4.6 et Sonnet 4.6 acceptent `max`, mais pas `xhigh`.

`/effort` enregistre les niveaux jusqu'à `xhigh` par modèle. **s** limite le choix à la session. `max` reste propre à la session ; `/effort auto` efface le choix enregistré pour le modèle actif.

`Option+T` / `Alt+T` bascule le thinking si le modèle le permet. Ce raccourci n'a pas d'effet sur Opus 5.5, Sonnet 5.5, Haiku 5.5 ni Fable, dont le thinking ne peut pas être désactivé. Réduire plutôt l'effort pour une tâche bornée.

`ultrathink` ajoute une consigne de réflexion sans changer l'effort API. `think hard` est du texte ordinaire.

Workflow typique

```
1. Démarrer la session   → claude
2. Vérifier le contexte  → /context
3. Plan Mode             → Shift+Tab × 2 (tâches complexes)
4. Décrire la tâche      → prompt clair et précis
5. Vérifier les diffs    → toujours lire avant d'accepter !
6. Accepter/Rejeter      → Entrée/Échap
7. Vérifier              → lancer les tests
8. Commit                → une fois la tâche terminée
9. /compact              → quand le contexte dépasse 70%
```

Formule de prompt rapide

```
QUOI : [Livrable concret]
OÙ : [Chemins de fichiers]
COMMENT : [Contraintes, approche]
VÉRIFIER : [Critères de succès]
```

Exemple :

```
Ajouter une validation d'entrée au formulaire de connexion.
OÙ : src/components/LoginForm.tsx
COMMENT : utiliser un schéma Zod, afficher les erreurs inline
VÉRIFIER : email vide affiche une erreur, format invalide affiche une erreur
```

Serveurs MCP

Serveur	Usage
Serena	Indexation + mémoire de session + recherche de symboles
grepai	Recherche sémantique + analyse du graphe d'appels
Context7	Documentation des librairies
Sequential	Raisonnement structuré
Playwright	Automatisation navigateur
Postgres	Requêtes base de données
doobidoo	Mémoire sémantique + Knowledge Graph

Mémoire Serena : `write_memory()` / `read_memory()` / `list_memories()`

Vérifier l'état : `/mcp`

Outils de recherche, aide-mémoire

Décision rapide : texte exact → `rg` | nom exact → `rg` / Serena | concept → grepai | structure → ast-grep

Tâche	Outil
Trouver du texte exact	`rg "TODO"`
Trouver du code lié à l'auth	`grepai search "authentication"`
Qui appelle une fonction	`grepai trace callers "login"`
Structure d'un fichier	`serena get_symbols_overview`

Aide-mémoire des flags CLI

Flag	Usage
`-p "requête"`	Mode non-interactif (CI/CD)
`-c` / `--continue`	Continuer la dernière session
`-r` / `--resume <id>`	Reprendre une session spécifique
`--teleport`	Téléporter une session depuis le web
`--model sonnet`	Changer de modèle
`--add-dir ../lib`	Autoriser l'accès hors du répertoire courant
`--permission-mode plan`	Plan mode
`--worktree` / `-w`	Lancer dans un git worktree isolé
`--max-budget-usd 5.00`	Plafond de dépense API (mode print)
`--dangerously-skip-permissions`	Auto-accepter (à manier avec précaution)
`--debug='mcp'`	Déboguer les serveurs MCP
`--allowedTools "Edit,Read"`	Autoriser ces appels sans nouvelle demande
`--tools "Edit,Read"`	Limiter les outils disponibles
`--debug`	Sortie de débogage

Anti-patterns

❌ Don't

- Prompts vagues
- Accepter sans lire
- Ignorer les avertissements
- Sauter les permissions
- Contraintes négatives uniquement

✅ Do

- Préciser fichier + ligne avec @références
- Lire chaque diff
- Utiliser /compact à 70%
- Jamais en production
- Fournir des alternatives

Optimisation des coûts

Modèles actuels

Tarifs API standard vérifiés le 6 octobre 2026. Dollars par million de tokens en entrée/sortie, hors cache.

Modèle	Entrée / sortie	Contexte	Effort par défaut
Haiku 5.5	$0,10 / $0,50 (jusqu'à 100K ; $0,50 / $2,50 au-delà)	1M	medium
Sonnet 5.5	$2 / $10	1M	medium
Opus 5.5	$4 / $20	1M	medium
Fable 5.1	$10 / $50	1M	high

Le tarif publié de Sonnet reste $2/$10 après la promotion de lancement. Opus 5.5 est le défaut du service direct, sauf règle du compte. Le modèle seul ne détermine pas le coût d'une tâche.

Modèle	Usage	Coût
Haiku	Corrections simples, revues	$
Sonnet	La majorité du développement	$$
Opus	Architecture, bugs complexes	$$$
OpusPlan	Plan (Opus) + Exécution (Sonnet)	$$

Arbre de décision rapide

Carte rapide des quatre couches

Couche	Question de choix	Référence
Modèle	Quel raisonnement et quel coût conviennent?	Glossaire
Runtime harness	Qui exécute la boucle outils-observation-décision?	Agent Harness Map
Repository harness	Quelles instructions, vérifications et états encadrent le dépôt?	Agent Harness Engineering
Orchestrateur	Faut-il coordonner plusieurs runtimes ou sessions?	Agent Tools

Évaluer le couple modèle-harness exact pour une tâche de code. Ajouter un orchestrateur lorsque la coordination est le besoin identifié. Un optimiseur de harness reste hors des quatre couches opérationnelles et modifie des harnesses candidats sous un protocole d'évaluation séparé.

Boucle et graphe ne sont pas des substituts de plus. La boucle nomme le feedback répété et la règle d'arrêt. Le graphe nomme les nœuds, branches et checkpoints exécutables dans la structure choisie. Pour les mécanismes observés, `confirmed` = mécanisme visible, `claimed` = déclaration source, `unknown` = point non établi. Voir aussi Loop & Graph Engineering.

```
Tâche simple        → demander directement à Claude
Tâche complexe      → planifier ; activer Tasks API si suivi persistant utile
Changement risqué   → Plan Mode d'abord
Tâche répétitive    → créer un agent ou une commande
Contexte plein      → /compact ou /clear
Besoin de doc       → utiliser Context7 MCP
Analyse profonde    → utiliser Opus (réflexion activée par défaut)
```

Les règles d'or

1. Toujours relire les diffs avant d'accepter
2. Utiliser /compact avant que le contexte devienne critique (>70%)
3. Être précis dans les demandes (QUOI, OÙ, COMMENT, VÉRIFIER)
4. Plan Mode d'abord pour les tâches complexes ou risquées
5. Créer un CLAUDE.md pour chaque projet
6. Commit fréquemment après chaque tâche terminée
7. Savoir ce qui est envoyé : prompts, fichiers, résultats MCP vers Anthropic

Résolution rapide des problèmes courants

Problème	Solution
« Command not found »	Vérifier le PATH et les instructions d'installation officielles
Contexte trop élevé (>70%)	`/compact` immédiatement
Réponses lentes	`/compact` ou `/clear`
MCP ne fonctionne pas	`claude mcp list` , vérifier la config
Permission refusée	Vérifier `settings.local.json`

Problème	Solution
Hook bloquant	Vérifier le code de sortie du hook, revoir la logique

Contrôle santé : `which claude && claude doctor && claude mcp list`

Auteur : Florian BRUNIAUX | Méthode Aristote | Écrit avec Claude

Version 3.44.1 | Octobre 2026